

From Spare Laptop to Sovereign BLAKE2b Miner

A "do it right" guide: your own Knots node + DATUM Gateway, ready before the hardware arrives

Status: DRAFT v0.1 — built from a real node build (Aug 30 – Sep 2, 2026). Sections marked ⌚ are not yet completed and will be filled in when the miners land.

Meet Genesis. The node in this guide is my old laptop. I named it Genesis because it was my starting point on the new chain: the first block of my own participation after the fork. Every "Genesis" below just means "the laptop."

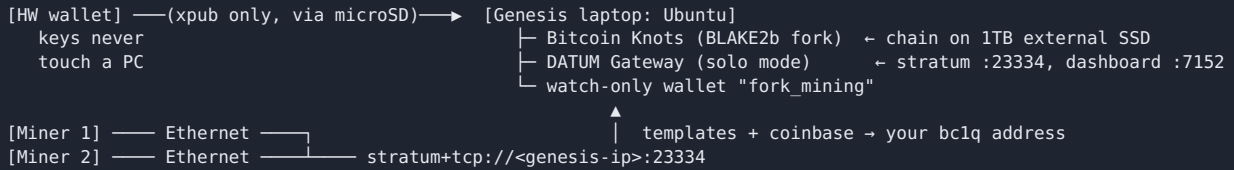
Who this is for: Anyone with a spare PC and a couple of used BLAKE2b ASICs who wants to mine to *their own node* instead of pointing at a Stratum v1 pool. If you can follow a recipe, you can do this in a weekend.

Who wrote it (and why that matters): I'm not a Linux or systems person. My background is SQL and analytics. Before this I had never partitioned a drive, built software from source, or written a systemd unit. I got Genesis running in a weekend by using Claude as a technical assistant the whole way: it read the official getting-started page, the Knots release notes, the DATUM README, and the router manual, and walked me through each step while I typed the commands and pasted back the errors. I mention this for two reasons. First, if you're on the fence because you're "not technical," that's not the barrier it used to be; the barrier is care, not skill. Second, the workflow in the box below is a big part of why this went smoothly, and it's worth copying.

How I used an AI assistant without getting burned - Sources first. I gave it the official pages (bitcoin-blake2b.org, bitcoinknots.org, the GitHub repos) and asked it to work from those, not from memory. When it wasn't sure, it said so and told me where to check. - **One step at a time.** I ran each command myself and pasted the exact output back before moving on. Most of the value was in it reading error messages I didn't understand. - **It never saw a secret.** No seed words, no private keys, no passwords. The wallet export it helped me massage contains public keys only. If a tool ever asks for a seed, the answer is no. - **I verified the things that matter myself.** GPG signatures, the payout address on the hardware wallet's own screen, the address in the DATUM config. Those checks are the whole point of "do it right," and they're the ones I didn't outsource.

Why bother: Knots forked to get away from large centralized pools. Every miner on an Sv1 pool is a vote for the problem we left behind. DATUM lets your miners build blocks from *your* node's template — you keep sovereignty, you keep 100% of blocks you find, and you're not someone else's hashrate.

0. The architecture in one picture



Three principles that drive every choice below:

Principle	What it means in practice
Keys off the network	A hardware wallet holds a <i>fresh</i> seed for this chain. Only the public descriptor ever touches the laptop.
Node is the source of truth	Light wallets can't see this chain (164-byte headers). Your full node is the only thing that can show your balance.
Headless and unattended	The node runs as a service in the background; it survives logouts, reboots, and power blips.

1. Bill of materials

Item	What Genesis used	Notes
Node machine	Spare Windows laptop, 32 GB RAM, 700 GB internal SSD	Beats a Raspberry Pi on every axis: x86 CPU for fast template building, RAM for a big <code>dbcache</code> , and the battery is a free UPS.
Chain storage	1 TB Samsung T7 external SSD	Full chain is ~700–750 GB. Keep the <code>datadir</code> on the external; internal drive just runs the OS.
Network	Ethernet to the router (TP-Link), port 8333 forwarded	Wi-Fi is the #1 cause of rejected shares. Run a cable.
Hardware wallet	Whatever I had handy, on a fresh seed dedicated to this chain, generated from my own entropy and verified on a second device	Any hardware wallet that exports a descriptor works. Don't trust any device's built-in RNG for the seed; supply your own entropy and confirm the result on a second, independent device.
Miners	2× iBeLink BM-S3 (19 TH/s each, used) ⌚	Antminer A3 and Goldshell SC-series also work. Older Sia-era gear is explicitly supported.
Power	Two dedicated 20 A / 240 V circuits in the utility room, L6-20R outlets, L6-20P-to-C19 cords into the miner PSUs	One miner per circuit. 240 V strongly preferred over 120 V.
Software	Ubuntu 24.04 Desktop, Bitcoin Knots 29.4.1.knots20260508 , CONVOY DATUM Gateway (BLAKE2b / header-v2 fork)	Exact sources in §4 and §6. Canonical docs: bitcoin-blake2b.org (bitcoin-blake2b.org redirects there).

2. Prepare the laptop (dual-boot Ubuntu alongside Windows)

You can wipe Windows entirely (simpler), but Genesis kept it. If you dual-boot:

In Windows first

1. Back up anything you care about. Partition resizing is routine but it's the one step with real data-loss potential.
2. Turn off **Fast Startup**: Control Panel → Power Options → "Choose what the power buttons do" → uncheck *Turn on fast startup*.
3. If **BitLocker** is on, suspend or decrypt it.
4. Shrink C: from **Disk Management** (right-click → Shrink Volume). Ubuntu needs ~60–80 GB. If Windows refuses to shrink enough, the culprit is usually the pagefile or hibernation file: disable both, reboot, retry.

Install Ubuntu

1. Download **Ubuntu 24.04 Desktop** (Desktop, not Server — its installer handles "alongside Windows" far more smoothly). Flash to USB with Balena Etcher or Rufus.
2. **Unplug the external chain drive before installing.** You do not want the installer anywhere near it.
3. Boot from USB (F12/Esc at power-on), choose *Install Ubuntu alongside Windows*, point it at the freed space. Create a user (Genesis uses `node`). Install OpenSSH when offered, or afterward: `sudo apt install -y openssh-server`.
4. First boot: `sudo apt update && sudo apt upgrade -y`

Make it behave like a server

1. Disable sleep/hibernate on lid close and on AC power (Settings → Power). This is the single most common "why did my node go offline."
2. If firmware supports a **battery charge limit**, cap it at 60–80 %. An always-full lithium battery on a 24/7 machine ages fast and swollen batteries are a fire risk.
3. Set GRUB's default to Ubuntu so an unattended reboot comes back as a node, not a Windows login screen. Know that major Windows updates occasionally overwrite GRUB; `boot-repair` from a live USB fixes it in minutes.
4. Give the laptop a **DHCP reservation** in your router so its IP never changes. (`ip addr` to find it.)

3. Mount the chain drive permanently

Format the external drive as ext4 if it's new (`sudo mkfs.ext4 /dev/sdX` , double-check the device name with `lsblk` first). If you already have chain data from another node, it can live here as-is.

```
sudo mkdir -p /mnt/chain
lsblk -f                # find the external drive's UUID
sudo nano /etc/fstab     # add the line below with YOUR uuid
```

```
UUID=<your-uuid> /mnt/chain ext4 defaults,nofail 0 2
```

```
sudo mount -a && ls /mnt/chain
```

nofail means the laptop still boots if the drive gets bumped loose. Genesis's datadir is `/mnt/chain/bitcoin`; create it with `mkdir -p /mnt/chain/bitcoin` and expect a multi-day initial sync on a fresh drive.

4. Install Bitcoin Knots (BLAKE2b release) and verify it

⚠ **Only** from bitcoinknots.org or github.com/bitcoinknots/bitcoin, release **v29.4.1.knots20260508**.

Anything else claiming to be "the fork client" is presumptively malware. **Never** type a seed phrase into anything during this process, and never use any "claim your fork coins" service.

1. Download the x86_64 Linux tarball plus `SHA256SUMS` and `SHA256SUMS.asc`.
2. Import the signers' keys. Genesis pulled them from github.com/bitcoinknots/guix.sigs (the site's TLS cert was expired at the time; the GitHub keys are canonical anyway). Genesis's build verified against six signers, including Luke and Mechanic personally.
3. Verify — do not skip this: `bash sha256sum --check SHA256SUMS --ignore-missing gpg --verify SHA256SUMS.asc SHA256SUMS`
4. Install: `bash tar xzf bitcoin-*.tar.gz sudo install -m 755 bitcoin-*/bin/* /usr/local/bin/bitcoind --version`

Configure `~/.bitcoin/bitcoin.conf`:

```
datadir=/mnt/chain/bitcoin
prune=0
dbcache=8192          # 32 GB box; use 4096 on 16 GB
server=1
# BLAKE2b activation headline — the value below is what Genesis runs with.
# Verify it against bitcoin-blake2b.org/getting-started for your release before trusting
mine.
blake2b_headline=8-30 NYPost Deride And Conquer
rpcbind=127.0.0.1
rpccallowip=127.0.0.1
rpcuser=<pick-a-user>
rpcpassword=<pick-a-long-password>
# tell DATUM the instant a new block arrives
blocknotify=curl -fsS -o /dev/null http://127.0.0.1:7152/NOTIFY
listenonion=0
```

Keep RPC on localhost. `txindex=1` is *not* needed for DATUM or `getblocktemplate`. The headline line is a public consensus parameter, not a secret, but it's the one line in this file you should copy from the official docs rather than from a stranger's guide (including this one).

First start — watch it in the foreground inside tmux:

```
tmux new -s node
bitcoind -printtoconsole
# detach: Ctrl+B then D      reattach: tmux attach -t node
```

If you're reusing chain data from a node that followed the legacy SHA-256 chain, the first start will **walk back to the fork point (961,640)** and download the BLAKE2b chain, which is tiny. A giant reorg in the log is *expected* on that one startup. If it complains of corruption: `-reindex-chainstate` (hours); worst case `-reindex` (about a day, no re-download).

Check progress from another window: `bitcoin-cli getblockchaininfo`

Confirm you're actually on the BLAKE2b rules:

```
bitcoin-cli getdeploymentinfo
# expect blake2b.height == 961640 and blake2b.active == true
```

5. Make it public infrastructure (optional but encouraged)

Forward **TCP 8333** on your router to the laptop's reserved IP. Make sure your modem is in bridge/passthrough mode (if the router's WAN IP matches your public IP, you're not double-NATed). Within hours you'll see `New inbound peers` in the log — you're now one of the nodes a stranger's fresh install can bootstrap from. Ten days after the fork, Genesis was serving **113 inbound peers** from a laptop on a kitchen counter. On a chain this young, that matters.

```
bitcoin-cli getpeerinfo | grep -c '"inbound": true'
```

Run as a service once you're happy, so it survives reboots:

```
/etc/systemd/system/bitcoind.service
```

```
[Unit]
Description=Bitcoin Knots (BLAKE2b fork)
After=network-online.target mnt-chain.mount
Requires=mnt-chain.mount

[Service]
User=node
ExecStart=/usr/local/bin/bitcoind
Restart=on-failure
TimeoutStopSec=600

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable --now bitcoind
```

(Stop the tmux instance first; two bitcoinds can't share a datadir. Same rule for `bitcoin-qt`: the GUI is nice for watching balances, but it dies when you log out. Keep the daemon headless and use `bitcoin-cli` or the DATUM dashboard for eyes-on.)

6. Build DATUM Gateway (BLAKE2b fork)

Ocean's stock DATUM doesn't speak the new header format. The official mining page (bitcoin-blake2b.org/mining) points at the **CONVOY** fork, which is the BLAKE2b / header-v2 DATUM. Genesis runs CONVOY. (During fork week I built from an earlier community fork that also had the critical fix, then rebuilt from CONVOY on Sept 10 so this guide matches what I actually run. The switch took ten minutes: clone, build, point it at the same config file, restart.)

```
sudo apt install -y cmake build-essential pkg-config libmicrohttpd-dev \
    libjansson-dev libcurl4-openssl-dev libsodium-dev git
git clone https://github.com/CONVOYMining/datum_gateway.git ~/datum_gateway
cd ~/datum_gateway
git log --oneline | grep 56c31f4      # MUST be present (or later): header-v2 bit in BLAKE2b
commitment
cmake . && make                      # follow the repo README if the build steps differ
```

Builds **without** commit 56c31f4 produce blocks whose hash disagrees with Knots: BLOCK FOUND followed by high-hash rejects. Check for it before you build.

Config (`datum_gateway_config.json`). This is the solo recipe from the official mining page; CONVOY's own example config is a *pooled* example and will not work for solo as-is.

```
{
  "bitcoind": { "rpcuser": "<user>", "rpcpassword": "<password>", "rpcurl": "http://
127.0.0.1:8332" },
  "stratum": { "listen_addr": "0.0.0.0", "listen_port": 23334 },
  "api":      { "listen_addr": "0.0.0.0", "listen_port": 7152, "admin_password": "<change-
me>", "modify_conf": true },
  "mining":   { "pool_address": "bc1q...your address from $7...", "coinbase_tag_primary":
"Genesis", "allow_hasher_time_rolling": false },
  "datum":    { "pool_host": "", "pooled_mining_only": false }
}
```

Field	Why it matters
<code>mining.pool_address</code>	Where every block you find pays. Verify it twice.
<code>datum.pool_host = ""</code> and <code>pooled_mining_only = false</code>	Both are required for solo. Empty the host but leave pooled-only true and the process just exits. "NON-POOLED MINING!" in the log is confirmation, not a warning.
<code>stratum.listen_port</code> 23334	What your miners connect to
<code>api.*</code> 7152	Web dashboard (digest auth with <code>admin_password</code>)
<code>api.modify_conf</code>	Lets you change <code>pool_address</code> from the dashboard without editing files

Genesis keeps the config at `~/datum_gateway_config.json`, outside the repo, so a rebuild or a second checkout can reuse it unchanged:

```
./datum_gateway -c ~/datum_gateway_config.json
```

A healthy start prints three things in the first second: the `NON-POOLED MINING!` banner, `API listening on ... port 7152`, and `Stratum V1 Server Init complete`, followed by `NEW NETWORK BLOCK` and a template line showing the block reward and transaction count. Run it in its own tmux window (or a second systemd unit once stable). Then close the loop with two checks:

1. **Dashboard:** `http://<genesis-ip>:7152` → Config → mining → confirm `pool_address` is yours.
2. **Template sanity:** `bash bitcoin-cli getblocktemplate '{"rules":["segwit","blake2b"]}' | grep -e '!blake2b' -e blake2b_headline` You should see `!blake2b` in the rules, proving Knots is advertising BLAKE2b work. CONVOY serves no jobs at all if it isn't.

Your gateway is now `stratum+tcp://<genesis-ip>:23334`, a fully working solo pool waiting for hardware.

7. The payout wallet: hardware-wallet keys, node eyes

The fork changed proof-of-work, not keys. Your existing hardware wallet's cryptography works unchanged (same bech32 `bc1q` addresses). What's broken is the *watching* layer, so the node does the watching.

1. **Dedicate a fresh seed** on a hardware wallet, labeled "BLAKE2b fork." **Provide your own entropy** (dice rolls are the standard way) rather than trusting the device's random number generator, and **verify the resulting seed on a second, independent device**: restore the words on a different wallet (or a trusted offline tool) and confirm the master fingerprint and first receive address match. If they don't, the seed is wrong; stop. Don't reuse your main stack's seed: mined coins are chain-exclusive and a clean seed keeps the bookkeeping absolute.
2. **Export the descriptor** (native SegWit) from the wallet's export menu to microSD. This contains public keys only.
3. **Create a watch-only wallet on the node** and import the descriptors: `bash bitcoin-cli createwallet "fork_mining" true true "" false true bitcoin-cli -rpcwallet=fork_mining importdescriptors '[ {"desc":"<receive wpkh descriptor with`

```
checksum>","timestamp":"now","active":true,"internal":false}, {"desc":"<change descriptor  
with checksum>","timestamp":"now","active":true,"internal":true}]' bitcoin-cli -  
rpcwallet=fork_mining getnewaddress "datum_payout" bech32
```

4. **Cross-check the address** on the hardware wallet's own screen before it goes anywhere near DATUM.
5. Paste it into DATUM's `pool_address` (dashboard or config), restart DATUM. From now on every block pays directly to keys that have never touched a computer.

Where Genesis actually is (Sept 10): still on a temporary *hot* wallet on the node itself, called `temp_mining`, created so the laptop's CPU could mine symbolically while the real hardware is in transit. It's a gesture more than an income strategy: a few hundred kilohashes against a network measured in petahashes will basically never find a block. But it means Genesis has been *mining*, not just validating, since the first week, and I think that matters. If you want the same:

```
bitcoin-cli createwallet "temp_mining"  
bitcoin-cli -rpcwallet=temp_mining getnewaddress "cpu_ceremony" bech32  
# in its own tmux window, grind real attempts at the current block:  
while true; do bitcoin-cli generatetoaddress 1 <that-address> 50000000; sleep 1; done
```

Hot-wallet rules: fine at a zero balance and fine for a ceremony, **not** where real miners should pay. Before the ASICs plug in, the hardware-wallet address goes into `pool_address` and the temp wallet gets retired (back it up first if it ever holds anything: `bitcoin-cli -rpcwallet=temp_mining backupwallet ~/temp_mining.dat`).

8. 🕒 Point the miners at Genesis (to be completed when the BM-S3s arrive)

Placeholders, to be replaced with screenshots and exact menu paths:

- Electrical: two dedicated 20 A / 240 V circuits, one per miner, L6-20R outlets with L6-20P-to-C19 cords straight into the PSUs. Miners are ~1.2 kW+ each and run 24/7; do not daisy-chain off a power strip.
- Network: Ethernet only. Assign each miner a DHCP reservation.
- Miner web UI → pool settings: URL `stratum+tcp://<genesis-ip>:23334`, worker `<any-name>`, password `x`. Leave pool 2/3 blank or point them at a small DATUM pool as failover (see §9).
- Confirm on the DATUM dashboard: worker connected, shares accepted, hashrate $\approx$ nameplate.
- Firmware notes for BM-S3 (fans, frequency, known quirks): TBD.
- Heat: plan the exhaust. In winter this offsets your furnace; in summer it's a problem to solve.

9. Failover and being a good citizen

Solo is variance-heavy. If you want smoother income, use a **DATUM-based pool** as your upstream rather than an Sv1 pool: the pool coordinates payouts but your node still builds the templates. Good citizens at time of writing self-cap their share of network hashrate; pick small ones. Never point at an Sv1 pool "just for failover": if your gateway hiccups, that's where your hash quietly goes.

10. Checklist: "ready before the miners show up"

#	Done when...	Genesis
1	Laptop boots to Ubuntu unattended, never sleeps	✓
2	Chain drive auto-mounts at /mnt/chain	✓
3	Knots BLAKE2b release installed, signatures verified	✓
4	Node at chain tip, <code>getblockchaininfo</code> shows current height	✓
5	Port 8333 forwarded, inbound peers appearing	✓ (113 as of Sept 10)
6	bitcoind running as systemd service	⌚ (still tmux, 7+ days uptime)
7	DATUM (CONVOY) built at 56c31f4 or later, solo mode, dashboard reachable	✓
8	Fresh hardware-wallet seed, watch-only wallet on node, address cross-checked	⌚
9	<code>pool_address</code> in DATUM = hardware-wallet address	⌚ (temp hot wallet, zero balance)
10	Something is hashing at the gateway (Genesis: CPU ceremony loop)	✓
11	Two 20 A / 240 V circuits with L6-20R outlets installed, Ethernet run to miner location	⌚
12	Miners hashing to :23334, dashboard shows accepted shares	⌚

Appendix: Handy commands

A companion file, **genesis-assistant-briefing.md**, packages this whole build as a structured runbook you can hand to your own AI assistant ("help me do this on my hardware"). It's the same steps, written for a machine to follow with you.

```
bitcoin-cli getblockchaininfo | grep -e blocks -e verificationprogress
bitcoin-cli getmininginfo      # network hashrate, difficulty
bitcoin-cli getpeerinfo | grep -c inbound
bitcoin-cli -rpcwallet=fork_mining getbalance
watch -n 5 sensors             # temps
tmux attach -t node
sudo systemctl status bitcoind
```

Draft by magicpete, an analytics guy who is not a sysadmin, with Claude as technical assistant reading the docs alongside me. Corrections from people who actually know what they're doing are very welcome. Nothing here is financial advice; mine because you believe in a decentralized network, and treat any coin value as upside.